

L6A: Sorting and Symbolic Processing

CS1101S: Programming Methodology

Sanka Rasnayaka

16th September, 2026

Outline

- ▶ Binary Search and Binary Search Trees (2.3.3)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.3>
- ▶ Sorting (not in SICPy)
- ▶ Symbolic Processing (2.3.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.2>
- ▶ Designing Data Structures (2.4)
<https://sourceacademy.nus.edu.sg/sicpy/2.4>

Announcements

- ▶ PA this Friday
 - Duration: 80 minutes
 - 2 sheets allowed (blank or printed)
 - No external keyboard/mouse
- ▶ Next week is Recess Week
- ▶ Midterm Assessment on Wednesday of Week 7 (30 Sept), 10am
- ▶ “The Choreographer Contest” closes today

Sumobot 2026: Only the best will win!

- ▶ October 15th, 6pm
- ▶ Dinner, prizes, swag, and a guest talk about Jane Street.
- ▶ Robot talent contest as well.



Outline

- ▶ Binary Search and Binary Search Trees (2.3.3)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.3>
- ▶ Sorting (not in SICPy)
- ▶ Symbolic Processing (2.3.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.2>
- ▶ Designing Data Structures (2.4)
<https://sourceacademy.nus.edu.sg/sicpy/2.4>

Game: Guess the Secret Number

- ▶ I have a secret number, an integer in the range $[1, 100]$
- ▶ You make a guess and I tell you whether your guess is “correct”, “too low”, or “too high”
- ▶ Your goal is to minimize the number of guesses
- ▶ What is your strategy?
 - Guess the “middle” number
 - If “too low”, continue to find in the “right” half
 - If “too high”, continue to find in the “left” half
 - Binary Search!!!

Guess the Secret Number using Binary Search

```
N = 100
...
def guess_secret_num(start, end):
    if start == end:
        return start
    else:
        guess = (math.floor((start + end) / 2))
        check = check_guess(guess)
        if check == "correct":
            return guess
        elif check == "too low":
            return guess_secret_num(guess + 1, end)
        else: # when "too high"
            return guess_secret_num(start, guess - 1)

guess_secret_num(1, N)
```

Observations

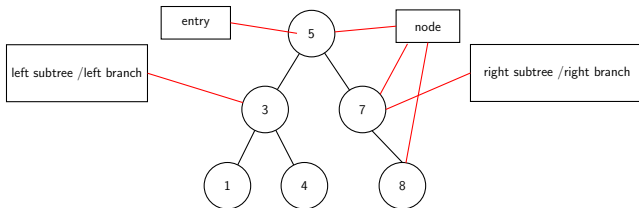
- ▶ Run time
 - At each step (wrong guess), we cut the search space in half
 - If problem size is $N = 2^k$, we get to size 1 after k steps
 - $k = \log_2 N$
 - Equivalent to $O(\log N)$
- ▶ Why does this work?
 - We need to assume that
 - all numbers “left” of the secret number are “too low”, and
 - all numbers “right” of the secret number are “too high”

Binary Search for Entries

- ▶ The problem
 - Check if an entry is included in a collection of entries
- ▶ Property of entries
 - A total order exists — two entries can be compared
 - They are “equal”, or one is either “smaller” or “larger” than the other
 - Examples: Numbers and Strings have such property
- ▶ Efficiency
 - Need to reach each “middle” entry in $O(1)$ time
 - May need a special data structure

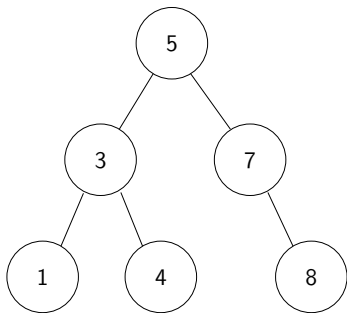
Binary Trees

- A binary tree is the empty tree, or it has
- an entry (which is the data item)
 - a left branch (called left subtree, which is a binary tree)
 - a right branch (called right subtree, which is a binary tree)



Binary Search Trees (BST)

- ▶ A binary search tree (BST) is a binary tree where
 - all entries in the left subtree are smaller than the entry, and
 - all entries in the right subtree are larger than the entry



A BST is an abstraction for binary search

Mission “Search and Rescue” (opens today)

- ▶ Given: A binary search tree (BST) of Strings
- ▶ Problem 1: Check if a given String occurs in BST
 - Use BST property to efficiently do so
- ▶ Problem 2: Add a new string to BST
 - Use BST property to efficiently do so
 - Preserve BST property

Outline

- ▶ Binary Search and Binary Search Trees (2.3.3)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.3>
- ▶ **Sorting (not in SICPy)**
 - The Problem of Sorting
 - Insertion Sort
 - Selection Sort
 - Merge Sort
- ▶ Symbolic Processing (2.3.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.2>
- ▶ Designing Data Structures (2.4)
<https://sourceacademy.nus.edu.sg/sicpy/2.4>

The Problem of Sorting

- ▶ Given:
 - A linked list xs of elements from a given universe X , and a total order on X
- ▶ Wanted:
 - A permutation of xs such that each element is greater than or equal to the previous one, with respect to the total order
- ▶ Comparisons only
 - The only allowed operations on the elements are comparisons, such as $<$, $>$, $<=$, $>=$, $==$ or $!=$

How to Sort a Llist with n Elements?

- ▶ Strategy:
 - Wishful thinking!
 - Imagine we can sort llists with m elements, where $m < n$
- ▶ Approach A
 - $m = n - 1$
- ▶ Approach B
 - $m = n / 2$

Approach A: Insertion Sort

► Idea:

- Sort the tail of the given llist using wishful thinking!
- Insert the head in the right place

► Implementation:

```
def insertion_sort(xs):  
    if is_none(xs):  
        return xs  
    else:  
        return insert(head(xs), insertion_sort(tail(xs)))
```

Insertion Sort

```
def insert(x, xs):
    if is_none(xs):
        return llist(x)
    elif x <= head(xs):
        return pair(x, xs)
    else:
        return pair(head(xs), insert(x, tail(xs)))

def insertion_sort(xs):
    if is_none(xs):
        return xs
    else:
        return insert(head(xs), insertion_sort(tail(xs)))
```

► Order of growth in time?

Approach A: Selection Sort

► Idea:

- Find the smallest element x and remove it from the llist
- Sort the remaining llist, and put x in front

► Implementation:

```
def selection_sort(xs):  
    if is_none(xs):  
        return xs  
    else:  
        x = smallest(xs)  
        return pair(x, selection_sort(remove(x, xs)))
```

Selection Sort

```
# find smallest element of non-empty llist xs
def smallest(xs):
    return reduce(lambda x, y: (x if x < y else y),
                  head(xs),
                  tail(xs))

def selection_sort(xs):
    if is_none(xs):
        return xs
    else:
        x = smallest(xs)
        return pair(x, selection_sort(remove(x, xs)))
```



► Order of growth in time?

Recall: How to Sort a Llist with n Elements?

► Strategy:

- Wishful thinking!
- Imagine we can sort llists with m elements, where $m < n$

► Approach A

- $m = n - 1$

► Approach B

- $m = n / 2$
- Idea: Split the llist in half, sort each half using wishful thinking, merge the sorted llists together

Approach B: Merge Sort

► Idea:

- Split the llist in half, sort each half using wishful thinking
- Merge the sorted llists together

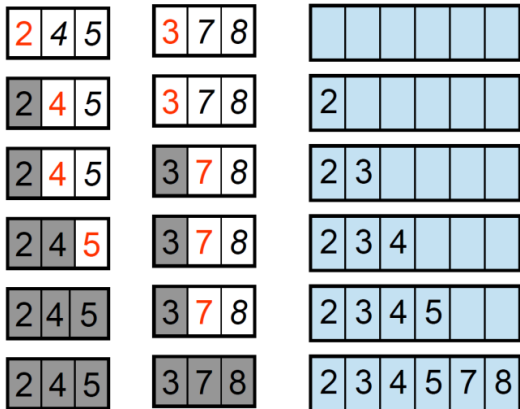
► Implementation:

```
def merge_sort(xs):  
    if is_none(xs) or is_none(tail(xs)):  
        return xs  
    else:  
        mid = middle(length(xs))  
        return merge(  
            merge_sort(take(xs, mid)),  
            merge_sort(drop(xs, mid))  
        )
```

Merge Sort (Function merge)

```
def merge(xs, ys):
    if is_none(xs):
        return ys
    elif is_none(ys):
        return xs
    else:
        x = head(xs)
        y = head(ys)
        if x < y:
            return pair(x, merge(tail(xs), ys))
        else:
            return pair(y, merge(xs, tail(ys)))
```

Merge Sort (Function merge): Example



Merge Sort (Other Helper Functions)

```
# half, rounded downwards  
def middle(n):  
    return math.floor(n / 2)
```

```
# put the first n elements of xs into an llist  
def take(xs, n):  
    # ???
```

```
# drop the first n elements from the llist  
# and return the rest  
def drop(xs, n):  
    # ???
```

Merge Sort

- ▶ Order of growth in time?
 - We will discuss this in Reflection R6

Approach B: QuickSort

- ▶ Question: Is there another way of “splitting” and “combining”?
- ▶ Answer: Yes! Details in Mission “Sorting Things Out”

Check-In time!

I'm not messy - I'm merely unsorted!

Look for Check-In I6A in the Source Academy!

Outline

- ▶ Binary Search and Binary Search Trees (2.3.3)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.3>
- ▶ Sorting (not in SICPy)
- ▶ Symbolic Processing (2.3.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.2>
 - Symbolic Evaluation
 - Symbolic Differentiation
- ▶ Designing Data Structures (2.4)
<https://sourceacademy.nus.edu.sg/sicpy/2.4>

Representing Functions: Directly

- ▶ Our first approach is to represent functions directly in Python using function definitions and lambda expressions
- ▶ Example: $f(x) = x^2 + 1$

```
def my_fun(x):  
    return x * x + 1  
  
print(my_fun(7))    # prints 50
```



Numerical Differentiation

$$f'(x) \approx \frac{f(x + dx) - f(x)}{dx}$$

```
# numerical differentiation; simplest method
def deriv_numeric(f):
    dx = 0.0001
    return lambda x: (f(x + dx) - f(x)) / dx
```

```
# Example use:
f = lambda x: x * x + x + 4
f_prime = deriv_numeric(f)

print(f(3))    # prints 16
print(f_prime(3)) # prints 7.000100000027487
```



Symbolic Evaluation

- ▶ Now, we represent functions / expressions with data structures
- ▶ Example expression:

$$my_exp = x^2 + x + 4$$

```
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))
```

- ▶ Symbolic evaluation:

```
eval_symbolic(my_exp, "x", 3)  
# should return 16
```

Definition of eval_symbolic

```
def eval_symbolic(exp, name, val):
    if is_number(exp):
        return exp
    elif is_variable(exp):
        if is_same_variable(exp, name):
            return val
        else:
            return math_nan
    elif is_sum(exp):
        return (eval_symbolic(addend(exp), name, val)
                + eval_symbolic(augend(exp), name, val))
    elif is_product(exp):
        return (eval_symbolic(multiplier(exp), name, val)
                * eval_symbolic(multiplicand(exp), name, val))
    else:
        return error(exp, "unknown expression type")
```

eval_symbolic example

```
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))  
  
print(eval_symbolic(my_exp, "x", 3))  
# prints 16
```



Symbolic Differentiation

► Symbolic representation:

```
# my_exp represents x * x + x + 4  
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))
```

► Symbolic differentiation:

$$f(x) = x^2 + x + 4$$
$$f'(x) = 2x + 1$$

```
deriv_symbolic(my_exp, "x")  
# should return make_sum(make_product("x", 2), 1)
```

```
eval_symbolic(deriv_symbolic(my_exp, "x"), "x", 3)  
# should return EXACTLY 7
```

Rules of Differentiation

► $\frac{dc}{dx} = 0$ for c a constant or a variable other than x

► $\frac{dx}{dx} = 1$

► $\frac{d(u + v)}{dx} = \frac{du}{dx} + \frac{dv}{dx}$

► $\frac{d(uv)}{dx} = u \frac{dv}{dx} + \frac{du}{dx} v$

Definition of deriv_symbolic

```
def deriv_symbolic(exp, x):
    if is_number(exp):
        return 0
    elif is_variable(exp):
        if is_same_variable(exp, x):
            return 1
        else:
            return 0
    elif is_sum(exp):
        return make_sum(
            deriv_symbolic(addend(exp), x),
            deriv_symbolic(augend(exp), x))
    elif is_product(exp):
        return make_sum(
            make_product(
                multiplier(exp),
                deriv_symbolic(multiplicand(exp), x)),
            make_product(
                deriv_symbolic(multiplier(exp), x),
                multiplicand(exp)))
    else:
        return error(exp, "unknown expression type")
```



Implementation of Symbolic Representation

► Constructors:

```
def make_sum(a1, a2):  
    return llist("+", a1, a2)  
  
def make_product(m1, m2):  
    return llist("*", m1, m2)
```

► This is a **tagged llist** representation

Implementation of Symbolic Representation

► Accessors / Selectors:

```
def addend(s):  
    return head(tail(s))  
  
def augend(s):  
    return head(tail(tail(s)))  
  
def multiplier(s):  
    return head(tail(s))  
  
def multiplicand(s):  
    return head(tail(tail(s)))
```

Implementation of Symbolic Representation

► Predicates:

```
def is_variable(x):  
    return is_string(x)  
  
def is_same_variable(v1, v2):  
    return (is_variable(v1) and is_variable(v2)  
            and v1 == v2)  
  
def is_sum(x):  
    return is_pair(x) and head(x) == "+"  
  
def is_product(x):  
    return is_pair(x) and head(x) == "*"
```

Revisiting Example

► Symbolic representation:

```
# my_exp represents  $x * x + x + 4$   
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))
```

► Symbolic differentiation:

$$f(x) = x^2 + x + 4$$

$$f'(x) = 2x + 1$$

```
print_llist(deriv_symbolic(my_exp, "x"))  
# should return make_sum(make_product("x", 2), 1)  
  
# but instead returns a complicated expression  
# equivalent to  $x * 1 + 1 * x + 1 + 0$ 
```



make_sum with Expression Simplification

```
def make_sum(a1, a2):
    if number_equal(a1, 0):
        return a2
    elif number_equal(a2, 0):
        return a1
    elif ((is_number(a1)) and (is_number(a2))):
        return a1 + a2
    elif (is_variable(a1) and is_variable(a2)
          and is_same_variable(a1, a2)):
        return make_product(2, a1)
    else:
        return llist("+", a1, a2)
```

make_product with Expression Simplification

```
def make_product(m1, m2):
    if number_equal(m1, 0) or number_equal(m2, 0):
        return 0
    elif number_equal(m1, 1):
        return m2
    elif number_equal(m2, 1):
        return m1
    elif ((is_number(m1)) and (is_number(m2))):
        return m1 * m2
    else:
        return llist(" * ", m1, m2)
```

Revisiting Example, Again

► Symbolic representation:

```
# my_exp represents x * x + x + 4  
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))
```

► Symbolic differentiation:

$$f(x) = x^2 + x + 4$$

$$f'(x) = 2x + 1$$

```
print_llist(deriv_symbolic(my_exp, "x"))  
# should return make_sum(make_product("x", 2), 1)  
  
# now returns an expression  
# equivalent to 2 * x + 1
```



Outline

- ▶ Binary Search and Binary Search Trees (2.3.3)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.3>
- ▶ Sorting (not in SICPy)
- ▶ Symbolic Processing (2.3.2)
<https://sourceacademy.nus.edu.sg/sicpy/2.3.2>
- ▶ Designing Data Structures (2.4)
<https://sourceacademy.nus.edu.sg/sicpy/2.4>
 - Specification
 - Implementation

Example: Symbolic Differentiation

- ▶ Example constructor:

```
def make_sum(a1, a2):  
    return llist("+", a1, a2)
```

- ▶ Example accessor / selector:

```
def addend(s):  
    return head(tail(s))
```

- ▶ Example predicate:

```
def is_sum(x):  
    return is_pair(x) and head(x) == "+"
```

Example: Symbolic Differentiation

- ▶ These functions
 - Provide an abstraction of the data structure
 - “Shield” the rest of the program from the implementation of the data structure
 - For example, `eval_symbolic` and `deriv_symbolic` do not need to know the actual implementation

Process of Designing Data Structures

- ▶ Specification: describes what is done (contract)
- ▶ Implementation: describes how it is done (work)

Recall: Rational Numbers

- ▶ Specification:
 - The ratio between a denominator and a numerator
- ▶ Constructor: `make_rat(n, d)`
- ▶ Selectors: `numer(x)`, `denom(x)`
- ▶ Using this abstraction, `add_rat(x, y)`, `sub_rat(x, y)`, `mul_rat(x, y)`, `div_rat(x, y)`, `equal_rat(x, y)` do not need to know the actual implementation

Recall: Rational Numbers

- ▶ Implementation:
 - Using `pair`
- ▶ Design Decisions
 - Head as the numerator and tail as the denominator
 - `make_rat`, could store given numerator and denominator, or divide by `gcd` to store in the simplest form
 - Store the sign separately in the data structure
- ▶ Consider alternatives and choose the appropriate design

Summary

- ▶ Binary search and binary search trees (BST)
- ▶ Sorting
 - Approach A: insertion sort, selection sort
 - Approach B: merge sort, quicksort
- ▶ Symbolic processing
 - Evaluation & differentiation
- ▶ Data structure design
 - Specification vs implementation
 - Consider design choices